ytLending购买抵押品文档

目录

1. [快速开始](#快速开始)
2. [脚本功能说明](#脚本功能说明)
3. [环境变量配置](#环境变量配置)
4. [执行流程](#执行流程)
5. [常见问题](#常见问题)

快速开始

代码块

```
# 最小配置运行（自动扫描所有资产）
export LENDING_ADDRESS="0x5FbDB2315678afecb367f032d93F642f64180aa3"
npx hardhat run scripts/utils/buyCollateral.ts --network sepolia

# 指定滑点
export LENDING_ADDRESS="0x5FbDB2315678afecb367f032d93F642f64180aa3"
export SLIPPAGE="2"
npx hardhat run scripts/utils/buyCollateral.ts --network sepolia
```

脚本功能说明

什么是购买清算抵押品?

当借贷协议的储备金低于目标储备金时，协议会开放清算抵押品的购买功能。用户可以用折扣价格购买协议中的抵押品资产，帮助协议补充储备金。

主要功能

1. 自动扫描所有抵押品资产

脚本通过遍历合约的 `assetList` 数组，自动发现所有已配置的抵押品地址，无需手动指定。对每个资产检查其 `collateralReserves`，跳过储备为零的资产。

2. 动态计算支付金额

无需预先指定 `BASE_AMOUNT`。每次购买前实时读取买家当前余额作为 `baseAmount` 上限传入合约。合约内部会将购买量自动限制到实际储备量，并通过 `quoteBaseAmount()` 只收取对应的实际费用，买家不会多付。

3. 基于储备量的滑点保护

```
代码块
minAmount = collateralReserves[asset] * (1 - slippage)
```

滑点作用于链上实际储备量，而非 quote 估算值，更准确地反映价格波动风险。默认滑点 1%，允许合理的价格偏移。

4. 多资产顺序购买，容错处理

- 逐资产执行，单个资产失败不影响其他资产
- 每次购买前重新读取买家余额，余额耗尽时提前退出并提示
- 最终输出所有资产的购买汇总

5. 一次性 MaxUint256 授权

如果授权额度低于 `MaxUint256 / 2`，自动执行一次 `approve(MaxUint256)`，后续所有资产购买无需重复授权。

环境变量配置

必需参数

变量名	说明	示例
`LENDING_ADDRESS`	Lending 合约地址	`0x5FbDB2315678afecb367f032d93F642f64180aa3`

可选参数

变量名	说明	默认值	取值范围
`SLIPPAGE`	滑点容忍度（百分比）	`1`	`0-10`，推荐 `1-2`

> **说明：** 原有的 `ASSET_ADDRESS` 和 `BASE_AMOUNT` 参数已移除。

> - 资产地址：由脚本自动从合约 `assetList` 扫描获取

> - 支付金额：由买家当前余额动态决定，合约只收取实际成本

配置示例

代码块

```
# 示例 1：最小配置，使用默认 1% 滑点
export LENDING_ADDRESS="0x5FbDB2315678afecb367f032d93F642f64180aa3"

# 示例 2：指定 2% 滑点（高波动市场）
export LENDING_ADDRESS="0x5FbDB2315678afecb367f032d93F642f64180aa3"
export SLIPPAGE="2"

# 示例 3：测试环境，较高滑点
export LENDING_ADDRESS="0x5FbDB2315678afecb367f032d93F642f64180aa3"
export SLIPPAGE="5"
```

执行流程

代码块

```
启动脚本
  └─ 读取环境变量，验证参数
        │
        ▼
  1. 初始化
     获取 Lending 合约、买家账户、baseToken 合约
        │
        ▼
  2. 系统状态检查
     检查 reserves < targetReserves
     不满足则退出
        │
        ▼
  3. 扫描可购买资产
     遍历合约 assetList[]
     过滤出 collateralReserves > 0 的资产
        │
        ▼
  4. 授权检查
     allowance < MaxUint256/2 ?
       是 → approve(MaxUint256)
       否 → 跳过
        │
```

```
          ▼
    5. 逐资产购买循环
       for each asset with reserves:
         ├─ 读取买家当前余额 buyerBalance
         ├─ 余额为 0 → break
         ├─ minAmount = reserves * (1 - slippage)
         ├─ buyCollateral(asset, minAmount, buyerBalance, buyer)
         │    合约内部:
         │      collateralAmount = min(quoteCollateral(buyerBalance), reserves)
         │      actualCost = quoteBaseAmount(collateralAmount)
         │      transferFrom(buyer, actualCost)  ← 只收实际费用
         └─ 解析 BuyCollateral 事件，输出结果
          │
          ▼
    6. 输出汇总
       成功购买资产数 / 总数
       累计支付 baseToken
       买家剩余余额
```

`getBuyCollateralInfo()` 辅助函数

该函数供外部调用，返回单个资产的购买预估信息：

返回字段	说明
`availableReserve`	链上可用抵押品储备量
`expectedAmount`	按 `baseAmount` 估算可购买数量（可能超过储备）
`actualAmount`	实际可购买数量（受储备限制后）
`minAmount`	应用滑点后的最小接受量
`actualBaseAmount`	估算实际需支付的 baseToken 数量
`isLimited`	是否受储备量限制

完整代码

代码块

```
import { ethers } from "hardhat";
import { Lending } from "../../typechain-types";

/**
 * 购买清算抵押品脚本
 *
 * 自动扫描合约中所有抵押品资产，对有储备的资产执行购买。
 * 传入买家当前余额作为 baseAmount 上限，合约自动按实际储备量收费。
 * 无需指定具体资产地址，脚本会自动遍历合约的 assetList。
 *
 * 环境变量:
 * - LENDING_ADDRESS: Lending 合约地址（必填）
 * - SLIPPAGE（可选）: 滑点容忍度百分比（1-5），默认 1
 */
async function main() {
  // ==================== 配置 ====================
  const LENDING_ADDRESS = process.env.LENDING_ADDRESS;
  const SLIPPAGE_PERCENT = parseFloat(process.env.SLIPPAGE || "1");

  if (!LENDING_ADDRESS || LENDING_ADDRESS === "0x...") {
    throw new Error("请设置 LENDING_ADDRESS 环境变量");
  }
  if (SLIPPAGE_PERCENT < 0 || SLIPPAGE_PERCENT > 10) {
    throw new Error("SLIPPAGE 应在 0-10 之间");
  }

  const SLIPPAGE = SLIPPAGE_PERCENT / 100;

  console.log("==================== 购买清算抵押品 ====================");
  console.log(`Lending 合约: ${LENDING_ADDRESS}`);
  console.log(`滑点容忍度: ${SLIPPAGE_PERCENT}%`);
  console.log("");

  // ==================== 初始化 ====================
  const lending = await ethers.getContractAt("Lending", LENDING_ADDRESS) as unknown as Lending;
  const [buyer] = await ethers.getSigners();
  const baseToken = await lending.baseToken();
  const base = await ethers.getContractAt("IERC20Metadata", baseToken);
  const baseDecimals = await base.decimals();

  console.log(`买家地址: ${buyer.address}`);

  // ==================== 系统状态检查 ====================
  console.log("\n检查系统状态...");
  const reserves = await lending.getReserves();
  const targetReserves = await lending.targetReserves();
```

```
  console.log(`当前储备金: ${ethers.formatUnits(reserves, baseDecimals)} baseToken`);
  console.log(`目标储备金: ${ethers.formatUnits(targetReserves, baseDecimals)} baseToken`);

  if (reserves >= 0n && BigInt(reserves.toString()) >= targetReserves) {
    throw new Error("储备金充足，当前无法购买抵押品");
  }

  // ==================== 扫描可购买资产 ====================
  const assetsToProcess = await getAllAssets(lending);
  console.log(`\n发现 ${assetsToProcess.length} 个抵押品资产`);

  // 过滤出有储备的资产
  const assetsWithReserves: { address: string; reserve: bigint; decimals: number }[] = [];
  for (const assetAddr of assetsToProcess) {
    const reserve = await lending.getCollateralReserves(assetAddr);
    if (reserve > 0n) {
      const assetToken = await ethers.getContractAt("IERC20Metadata", assetAddr);
      const dec = await assetToken.decimals();
      assetsWithReserves.push({ address: assetAddr, reserve, decimals: dec });
      console.log(`  ${assetAddr}: 储备 ${ethers.formatUnits(reserve, dec)} 代币`);
    }
  }

  if (assetsWithReserves.length === 0) {
    console.log("\n所有资产储备均为零，无需购买。");
    return;
  }

  // ==================== 授权（一次性 MaxUint256）====================
  console.log("\n检查授权...");
  const allowance = await base.allowance(buyer.address, LENDING_ADDRESS);
  if (allowance < ethers.MaxUint256 / 2n) {
    console.log("正在授权 MaxUint256...");
    const approveTx = await base.approve(LENDING_ADDRESS, ethers.MaxUint256);
    await approveTx.wait();
    console.log("授权成功");
  } else {
    console.log("授权充足，无需重复授权");
  }

  // ==================== 逐资产购买 ====================
  let totalPaid = 0n;
```

```
  let successCount = 0;

  for (const { address: assetAddr, reserve, decimals: assetDecimals } of assetsWithReserves) {
    console.log(`\n---- 购买资产: ${assetAddr} ----`);

    // 读取买家当前余额作为本次最大支付额
    const buyerBalance = await base.balanceOf(buyer.address);
    if (buyerBalance === 0n) {
      console.log("买家余额已耗尽，跳过剩余资产");
      break;
    }
    console.log(`买家当前余额: ${ethers.formatUnits(buyerBalance, baseDecimals)} baseToken`);
    console.log(`可用储备: ${ethers.formatUnits(reserve, assetDecimals)} 代币`);

    // minAmount = 储备量 * (1 - slippage)，允许价格轻微偏移
    const slippageMultiplier = BigInt(Math.floor((1 - SLIPPAGE) * 1e18));
    const minAmount = (reserve * slippageMultiplier) / BigInt(1e18);
    console.log(`最小接受量 (${SLIPPAGE_PERCENT}% 滑点): ${ethers.formatUnits(minAmount, assetDecimals)} 代币`);

    // 以买家全部余额作为 baseAmount 上限；合约内部按实际储备量收费
    try {
      const tx = await lending.buyCollateral(
        assetAddr,
        minAmount,
        buyerBalance,
        buyer.address
      );
      console.log(`交易已提交: ${tx.hash}`);
      const receipt = await tx.wait();
      console.log(`交易确认，Gas 消耗: ${receipt?.gasUsed.toString()}`);

      // 解析事件
      const buyEvent = receipt?.logs.find((log: any) => {
        try { return lending.interface.parseLog(log)?.name === "BuyCollateral"; }
        catch { return false; }
      });

      if (buyEvent) {
        const parsed = lending.interface.parseLog(buyEvent);
        const paidAmount: bigint = parsed?.args.baseAmount;
        const receivedAmount: bigint = parsed?.args.collateralAmount;
        totalPaid += paidAmount;
        successCount++;
```

```

        console.log(`实际支付: ${ethers.formatUnits(paidAmount, baseDecimals)} baseToken`);
        console.log(`实际获得: ${ethers.formatUnits(receivedAmount, assetDecimals)} 代币`);

        // 折扣信息
        const marketAmount = await lending.quoteCollateral(assetAddr, paidAmount);
        if (receivedAmount > marketAmount && marketAmount > 0n) {
          const discount = ((receivedAmount - marketAmount) * 10000n) / marketAmount;
          console.log(`折扣收益: +${ethers.formatUnits(receivedAmount - marketAmount, assetDecimals)} 代币 (${Number(discount) / 100}%)`);
        }
      }
    } catch (err: any) {
      console.log(`跳过 ${assetAddr}: ${err.message?.split("\n")[0] ?? err}`);
    }
  }

  // ==================== 汇总 ====================
  console.log("\n==================== 购买汇总 ====================");
  console.log(`成功购买资产数: ${successCount} / ${assetsWithReserves.length}`);
  console.log(`累计支付: ${ethers.formatUnits(totalPaid, baseDecimals)} baseToken`);

  const finalBalance = await base.balanceOf(buyer.address);
  console.log(`买家剩余余额: ${ethers.formatUnits(finalBalance, baseDecimals)} baseToken`);
  console.log("====================================================");
}

/**
 * 遍历合约 assetList 数组，获取所有抵押品地址
 */
async function getAllAssets(lending: Lending): Promise<string[]> {
  const assets: string[] = [];
  let i = 0;
  while (true) {
    try {
      const asset = await (lending as any).assetList(i);
      assets.push(asset);
      i++;
    } catch {
      break; // 数组越界，遍历结束
    }
```

```
  }
  return assets;
}

main()
  .then(() => process.exit(0))
  .catch((error) => {
    console.error("\n执行失败:", error.message || error);
    process.exit(1);
  });

/**
 * 获取单个资产的购买详情（供外部调用）
 */
export async function getBuyCollateralInfo(
  lendingContract: Lending,
  asset: string,
  baseAmount: bigint,
  slippageTolerance: number = 0.01
) {
  const availableReserve = await lendingContract.getCollateralReserves(asset);
  // minAmount 基于实际储备量而非 quote，允许 slippage 偏移
  const slippageMultiplier = BigInt(Math.floor((1 - slippageTolerance) * 1e18));
  const minAmount = (availableReserve * slippageMultiplier) / BigInt(1e18);

  // 用于展示：预估 baseAmount 能买到多少（可能超过储备，合约会自动限制）
  const expectedAmount = await lendingContract.quoteCollateral(asset, baseAmount);
  const actualAmount = expectedAmount < availableReserve ? expectedAmount : availableReserve;
  const actualBaseAmount = actualAmount < expectedAmount
    ? (baseAmount * actualAmount) / expectedAmount
    : baseAmount;

  return {
    availableReserve,
    expectedAmount,
    actualAmount,
    minAmount,
    baseAmount,
    actualBaseAmount,
    isLimited: actualAmount < expectedAmount,
  };
}
```

常见问题

Q1: 什么时候可以购买抵押品?

当协议储备金低于目标储备金时才开放购买:

```
代码块
1   可购买条件: reserves < targetReserves
```

可通过以下方式检查:

```
代码块
1   const reserves = await lending.getReserves();
2   const targetReserves = await lending.targetReserves();
3   console.log(`可以购买: ${reserves < targetReserves}`);
```

Q2: 滑点应该设置多少?

市场状况	建议滑点
正常市场	`1` （默认）
轻微波动	`2`
高波动市场	`3-5`
测试环境	`5` 或更高

滑点保护的含义：如果实际到手的抵押品数量低于 `minAmount` （储备量的 `1 - slippage` 倍），交易会 revert。

Q3: 如果某个资产购买失败会怎样?

单个资产失败（如该资产储备金刚好被其他人买走、价格剧烈波动超出滑点）不会中断整个脚本，会打印错误信息后继续处理下一个资产。

Q4: 购买的折扣是如何计算的?

折扣通过清算因子（`liquidationFactor`）和价格因子（`storeFrontPriceFactor`）计算:

```
代码块
discountFactor = storeFrontPriceFactor × liquidationFactor
effectiveAssetPrice = assetPrice × (1 - discountFactor)
```

示例：

```
代码块
assetPrice            = $2.00
liquidationFactor     = 0.05  (5%)
storeFrontPriceFactor = 0.90  (10%)
discountFactor        = 0.90 × 0.05 = 0.045
effectivePrice        = $2.00 × (1 - 0.045) ≈ $1.91  (折扣约 4.5%)
```

脚本会在每笔交易完成后自动计算实际折扣并输出。